Introduction to Functional Programming :

From JDK 1.8V onwards, Java also concentrated on function/method and introduced Functional Programming.

It is mainly used to write concise coding so the length of the method will be reduced.

What is a Functional Interface in java?

* If an interface **contains exactly one abstract method then it is called Functional Interface.**
Example :
public interface Drawable
{
void draw();// SAMM -> Single Abstract method
}

* A Functional Interface may contain 'n' **number of default and static methods** but it must contain exactly one abstract method.
* In order to **restrict the developer** to accept only one abstract method we can use @FunctionalInterface annotation before the interface name.
Example :
@FunctionalInterface
interface Drawable
{
void draw();
public String toString();
public boolean equals(Object obj);
public int hashCode();
}

//Program on Functional Interface :
@FunctionalInterface
interface Drawable
{
void draw();
public String toString();
public boolean equals(Object obj);
public int hashCode();
}

class Test
{
public static void main(String[] args)
{
Drawable d1 = new Drawable()
{
@Override
public void draw()
{
SO.println("Drawing something!!!");
}
};
}

d1.draw();
}
}

***** What is Lambda Expression in Java :**

* It is a new feature which is introduced from JDK 1.8V.

* It is used to enable **"Functional Programming"** in Java so, we can **concise** our code as well as we can remove **boilerplate** code.

* It is an anonymous function (**function without any name**) which does not need the following :
a) Name of the function/method
b) access modifier of the function/method
c) return type of the function/method
d) Type of the variable is also not required.

Case 1 :
Concrete Method :
public void show()
{
SO.println("Show method");
}

Converting to Lambda Expression :
() -> SO.println("Show method");

Case 2 :
Concrete Method :
public void doSum(int i, int y)
{
SO.println(i+y);
}

Lambda Expression :
(i, y) -> SO.println(i+y);

Case 3 :
Concrete Method :
public int getLength(String str)
{
return str.length();
}

Lambda Expression :
str -> str.length();

Anonymous Inner class VS Lambda Expression :

Anonymous

```
@FunctionalInterface
public interface Printable
{
void print();
}

public class Main
{
public static void main(String -a)
{
Printable p = new Printable()
{
@Override
public void print()
{
SO.println("Printing")
}
};
p.print();
}
}
```

Boiler plate
Nested Code

Lambda

```
@FunctionalInterface
public interface Printable
{
void print();
}

public class Main
{
public static void main(String -a)
{
Printable p = () -> SO.println("Printing");
p.print();
}
}
```

Line body GC

Lambda will work only with functional interface.

If the body of the Lambda Expression contains only one statement, then curly braces are optional.

We can also remove the variables type while defining the Lambda Expression parameter.

If the lambda expression method contains only one parameter, then we can remove () similar also.

In lambda expression return keyword is optional but if we use return keyword then {} are compulsory.

Independently Lambda Expression is not a statement.

Lambda target must be a functional interface only.

Lambda target can't be class or abstract class, it will work with functional interface only.

In the Lambda expression, we can use any type of variables (i.e. class level as well as
Method level but if we use local variable then it must be final OR effectively final.

//Programs :
package com.rvd.lamda_expressions;

```
interface Runnable
{
void run();
}

public class LambdaDemo
{
public static void main(String[] args)
{
Runnable car = () -> SO.println("Car is running");
CP.join();
Runnable bus = () -> SO.println("Bus is running");
bus.start();
Runnable bike = () -> SO.println("Bike is running");
bike.start();
}
}
```

package com.rvd.lamda_expressions;

```
@FunctionalInterface
interface Calculator
{
void Actual(int i, int y);
}

public class LambdaDemo2
{
public static void main(String[] args)
{
Calculator calc = (i, d) -> SO.println("Sum is "+(i+d));
calc.Actual(5, 10);
}
}
```

package com.rvd.lamda_expressions;

```
@FunctionalInterface
interface Length
{
int getLength(String str);
}

public class LambdaDemo3
{
public static void main(String[] rvd)
{
Length length = str -> str.length();
int
{
return str.length();
}
String sum = "Hi,Hello";System.out.Sum ("";
SO.println(sum.getLength());
}
}
```

package com.rvd.lamda_expressions;

```
@FunctionalInterface
interface Drawable
{
void draw();
}

public class LambdaDemo4
{
public static void main(String[] args)
{
String shapeType = "Circle";// check to final OR effectively final
if(shapeType = "Square")
{
Drawable d = s -> SO.println("Drawing Square");
d.draw();
}
}
}
```

package com.rvd.lamda_expressions;

```
@FunctionalInterface
interface Runnable
{
void run();
}

public class LambdaDemo5 {
public static void main(String[] args)
{
getRunnableObject().run();// Invoking run()
}

public static Runnable getRunnableObject() {// returning Runnable interface as a parameter
return
{
run();
}
};
}
}
```

The target of lambda Expression must be a functional interface.
interface Printable
{
void print();
}

class Test
{
public static void main(String[] args)
{
Printable p = () -> SO.println("Printing");
p.print();
}
}